ECE 447 Project

Kavin Kumaran, Adil Tapal, Robin Liu, Hasan Zia
University of Alberta, Department of Electrical and Computer Engineering
116 St & 85 Ave, Edmonton, AB T6G 2R3
kavinvas@ualberta.ca, atapal@ualberta.ca, robin9@ualberta.ca, hasanfar@ualberta.ca

1 Introduction

The central challenge in image classification is the fact that the same image can appear in different orientations, positions, and reflections. A handwritten digit "6" rotated 90° is still a "6," yet a standard convolutional neural network (CNN) must learn this equivalence from data, needing large quantities of training examples to execute this reliably.

Cohen and Welling [1] address this shortcoming directly. Their paper introduces *Group Equivariant Convolutional Networks* (G-CNNs), a generalisation of standard CNNs that explicitly encodes the symmetries of a task into the architecture itself. When we replace the standard planar convolution with a *group convolution* shown over a chosen symmetry group G, the network will become equivariant to all transformations in G by design instead of memorization. Now networks can share weights across spatial positions (like ordinary convolutions) and across orientations and reflections, which increases the effective degree of parameter sharing.

In this report, we can explain the core idea is the paper on top of the mathematical G-CNNs. We can begin with a brief overview of convolutional neural networks and group theory separately, which provides the necessary background to understand the G-convolution operation. We are then able to describe our reproduction experiments, offer a critical evaluation of the paper, and close with individual reflections from the group member.

2 What is a convolutional neural network?

A convolutional neural network (CNN) is a specific kind of neural network. In classical machine learning, models are trained on input data to then make predictions on similar data in the future. To train such models, one must manually determine what is deemed interesting - these are referred to as the features of the data. Neural networks aim to automate the feature extraction process [2]. When a neural network is given data, the data is put through different layers of computational neurons, which apply functions on the data. The specific functions used differ based on the type of neural network being used. This is where convolutional neural networks come in - the layers found within CNNs are inspired by the visual pathways within the human brain. As one may assume, CNNs are thus often used in computer vision contexts, to analyse and classify different images. Moreover, the functions of the layers include convolution/correlation, pooling, batch normalisation, flattening, and softmax [2]. For the sake of our discussion, we focus primarily on convolution/correlation and a specific form of pooling known as max-pooling. Informally, convolution applies a sort of filter (a weighted average) on a number of pixels to combine them into one "blurred" pixel. If $l \in \mathbb{N}$ is the layer, and the set of feature maps are of the form $f : \mathbb{Z}^2 \to \mathbb{R}^{K^l}$, a convolution will convolve them with a set of K^{l+1} filters of the form $\psi^i : \mathbb{Z}^2 \to \mathbb{R}^{K^l}$, which is formally expressed as the following function:

$$\mathbb{Z}^2 \to \mathbb{R}, x \mapsto [f * \psi^i](x) := \sum_{y \in \mathbb{Z}^2} \sum_{k=1}^{K^l} f_k(y)\psi^i(x-y).$$

Notice that the input to the filters is reversed (the input of the feature maps is y, but for the filters, it is x shifted by the inverse [i.e., negative] of y). Correlation is defined similarly, with the only difference

being that the input is not reversed:

$$\mathbb{Z}^2 \to \mathbb{R}, x \mapsto [f \star \psi^i](x) := \sum_{y \in \mathbb{Z}^2} \sum_{k=1}^{K^l} f_k(y)\psi^i(y-x).$$

Pooling also reduces the number of pixels, but instead of requiring filter functions, it is done solely with feature maps. Max-pooling, in particular, finds the maximum value of a given feature map over a so-called pooling domain, which is obtained from the input. It is defined as follows: suppose $U \subset \mathbb{Z}^2$ is a pooling domain, the set $n + U = \{n + u \,|\, n \in \mathbb{Z}^2, u \in U\}$ is a translation of U, and $f : \mathbb{Z}^2 \to \mathbb{R}$ is a feature map. Then, the max-pooling operation is given by some pooling operator P such that

$$Pf(n) := \max_{k \in (n+U)} f(k).$$

In other words, max-pooling finds the greatest value of a feature map f over some translation of a pooling domain $n + U$. Moreover, notice that these functions are defined with the domain of $\mathbb{Z}^2$. As will be explained in more detail later, this set can be thought of as being made up of 2D translations. As such, regular CNNs are able to correctly classify translated versions of images. However, they require additional training on rotated/reflected images (this is referred to as data augmentation). Cohen and Welling address this issue by generalising $\mathbb{Z}^2$, turning to concepts from a field in mathematics known as group theory.

3 What is a group?

A group is an algebraic structure. Formally, it is a tuple $G = (S, \circ)$ (where S is a set and $\circ : S \times S \to S$ is a binary operation) that satisfies the following axiomatic properties:

1. S is closed under $\circ$, i.e. for every $s_1, s_2 \in S$, we have that $s_1 \circ s_2 \in S$;
2. $\circ$ is associative, i.e. for every $s_1, s_2, s_3 \in S$, we have that $(s_1 \circ s_2) \circ s_3 = s_1 \circ (s_2 \circ s_3)$;
3. There exists an identity element $e \in S$ such that for every $s \in S$, we have that $e \circ s = s \circ e = s$; and
4. For every $s \in S$, there exists an inverse element $s^{-1} \in S$ such that we have $s \circ s^{-1} = s^{-1} \circ s = e$ [3].

While this is the proper definition, this may appear daunting and unintuitive - what sort of structure actually is a group, and what makes groups useful? A more intuitive description of groups would be that they describe sets of symmetries. They represent the different actions that may be taken on some object which leave the object looking the same. For example, consider a non-square rectangle. With this more general notion of a symmetry, we consider the following to be the symmetries of a non-square rectangle (which is known as the Klein four-group K_4):

1. Doing nothing (this is the identity symmetry e);
2. A $180°$ counter-clockwise rotation;
3. The vertical reflection; and
4. The horizontal reflection.

While a proof will not be given, all groups satisfying the formal definition may be expressed equivalently as a set of symmetries. (This follows due to a result in group theory known as Cayley's Theorem [3].) Moreover, this description illuminates the significance of groups - group theory allows us to formally study the behaviour of symmetries. As such, groups naturally may be used when training convolutional neural networks on understanding and classifying images. In particular, they allow us to incorporate rotations and reflections in the domains of our functions, removing the need to use data augmentation to account for these symmetries.

4 The groups $(\mathbb{Z}^2, +)$, $p4$, and $p4m$

Cohen and Welling discuss three particular groups: $(\mathbb{Z}^2, +)$, $p4$, and $p4m$. Informally, $(\mathbb{Z}^2, +)$ is the group of all 2D integer translations; $p4$ is the group of 2D integer translations along with all

90° rotations about any centre of rotation; and $p4m$ is the group of 2D integer translations, 90° rotations about any centre of rotation, and mirror reflections. As discussed above, $\mathbb{Z}^2$ is the domain for the convolution/correlation and max-pooling functions used by traditional CNNs. To incorporate rotations, $p4$ is introduced. Elements of this group are parametrised by the following function:

$$\{0,1,2,3\} \times \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}^{3\times 3}, (r,u,v) \mapsto g(r,u,v) := \begin{bmatrix} \cos\left(\frac{r\pi}{2}\right) & -\sin\left(\frac{r\pi}{2}\right) & u \\ \sin\left(\frac{r\pi}{2}\right) & \cos\left(\frac{r\pi}{2}\right) & v \\ 0 & 0 & 1 \end{bmatrix}.$$

As a group, $p4$ includes a group binary operation, which in this case is matrix multiplication. It may be verified that the group axioms mentioned in section 3 are satisfied by $p4$. Moreover, suppose $(u', v') \in \mathbb{Z}^2$ represents the position of a pixel. Then, an element in $p4$ may act on (u', v') as follows:

$$\mathbb{Z}^2 \to \mathbb{Z}^2, (u', v') \mapsto h\left(\begin{bmatrix} \cos\left(\frac{r\pi}{2}\right) & -\sin\left(\frac{r\pi}{2}\right) & u \\ \sin\left(\frac{r\pi}{2}\right) & \cos\left(\frac{r\pi}{2}\right) & v \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} u' \\ v' \\ 1 \end{bmatrix}\right),$$

where h is the projection function given by

$$\mathbb{Z}^3 \to \mathbb{Z}^2, (x,y,z) \mapsto h(x,y,z) := (x,y).$$

$p4m$ further extends to include reflections. Similarly to $p4$, we may parametrise elements of $p4m$, which is given by

$$\{0,1\} \times \{0,1,2,3\} \times \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}^{3\times 3}, (m,r,u,v) \mapsto g(m,r,u,v) := \begin{bmatrix} (-1)^m \cos\left(\frac{r\pi}{2}\right) & -(-1)^m \sin\left(\frac{r\pi}{2}\right) & u \\ \sin\left(\frac{r\pi}{2}\right) & \cos\left(\frac{r\pi}{2}\right) & v \\ 0 & 0 & 1 \end{bmatrix}.$$

Again, the group binary operation is matrix multiplication; this satisfies the group axioms mentioned previously. Once more, if $(u', v') \in \mathbb{Z}^2$ represents the position of a pixel, then an element of $p4m$ acts on (u', v') as follows:

$$\mathbb{Z}^2 \to \mathbb{Z}^2, (u', v') \mapsto h\left(\begin{bmatrix} (-1)^m \cos\left(\frac{r\pi}{2}\right) & -(-1)^m \sin\left(\frac{r\pi}{2}\right) & u \\ \sin\left(\frac{r\pi}{2}\right) & \cos\left(\frac{r\pi}{2}\right) & v \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} u' \\ v' \\ 1 \end{bmatrix}\right).$$

5 Summary of Cohen and Welling (2016)

5.1 The core idea: equivariance and invariance

The central idea behind Cohen and Welling is that standard CNNs are equivariant only to translations, and this specific limitation causes them to waste parameters showing that this can learn many copies of the same filter at different orientations. A map Φ is said to be equivariant with respect to a group G if

$$\Phi(T_g x) = T'_g \Phi(x) \quad \forall\, g \in G,$$

where T_g and T'_g are the actions of the group element g on the input and output spaces respectively. Intuitively, equivariance means that transforming the input and then applying the map gives the same result as applying the maps first and then transforming the output the transformation then passes through the layer. A similar concept (which is in fact a special case of equivariance) is invariance. A map Φ is said to be invariant with respect to a group G if

$$\Phi(T_g x) = \Phi(x) \quad \forall\, g \in G,$$

Notice that, if we replace T'_g in the definition of equivariance with T'_e (where e is the identity element, which corresponds to doing nothing), we obtain invariance. Moreover, as Cohen and Welling describe, equivariance is preferred over invariance for deep learning, as equivariance keeps track of the spatial configuration of the features [1].

Notice that, as previously described, traditional CNNs do not require data augmentation to correctly classify images that translate certain features that they have learnt. This is because CNNs are translation-equivariant. In particular, we have that

$$\Phi(T_n x) = T'_n \Phi(x) \quad \forall\, n \in \mathbb{Z}^2,$$

i.e. transformations corresponding to elements of $\mathbb{Z}^2$ (which would now be translations) commute with the map Φ. Cohen and Welling extend this property of equivariance from translations to rotations and reflections, which then allow us to avoid data augmentation for the purposes of recognising said transformations.

5.2 The G-convolution

Since $p4$ and $p4m$ are generalisations of $\mathbb{Z}^2$, the convolution/correlation and max-pooling functions described in section 2 must be modified (generalised from the domains of $\mathbb{Z}^2$ to arbitrary groups). We begin with the G-equivariant correlation function. Let $(G, \circ)$ be an arbitrary group. In the first layer, we have

$$G \to \mathbb{R}, g \mapsto [f \star \psi](g) := \sum_{y \in \mathbb{Z}^2} \sum_k f_k(y) \psi_k(g^{-1} y).$$

Here, the input data is coming directly from the image, hence why we may assume that $y \in \mathbb{Z}^2$, since the position of pixels in the image is given by elements of $\mathbb{Z}^2$. As a result, f and ψ are functions over $\mathbb{Z}^2$. However, as can be seen from the definition, $f \star \psi$ is defined for general groups, and this is necessary for all layers beyond the first. Thus, for all other layers, a more general definition is given:

$$G \to \mathbb{R}, g \mapsto [f \star \psi](g) := \sum_{h \in G} \sum_k f_k(h) \psi_k(g^{-1} h).$$

Notice that, for the input of the filters, we apply the group binary operation between g^{-1} and the input to $f \star \psi$. This is a generalisation of the standard definition, since the inverse of some $n \in \mathbb{Z}^2$ is $-n$ ($n + (-n) = (-n) + n = 0$; it can be checked that 0 is the identity element of $(\mathbb{Z}^2, +)$). Moreover, Cohen and Welling derive the equivariance of both of these functions, showing in particular that they commute with translations (in the case of the G-correlation in the first layer), and with all symmetries in G (in the case of the correlation for all other layers). This is because, for the first layer, f and ψ are functions over $\mathbb{Z}^2$, so any transformations that may be applied on the two must also be over $\mathbb{Z}^2$. For every other layer, f and ψ are functions over G, so the transformations are now more generally over G as well.

5.3 Other layers and full G-CNN equivariance

Aside from convolution/correlation, some more functions are defined for G-CNNs. These include nonlinearities and subgroup and coset max-pooling. We begin with nonlinearities. First, they address issues that may arise due to nonlinearities. If $\nu : \mathbb{R} \to \mathbb{R}$ is a nonlinearity, then we have the corresponding composition operator C_ν, defined as

$$C_\nu f(g) := [\nu \circ f](g) = \nu(f(g)).$$

It is shown that $C_\nu L_h f = L_h C_\nu f$, i.e. C commutes with an arbitrary left transformation L. In other words, the composition operator is equivariant to all group transformations. Next, we discuss subgroup and coset max-pooling. Max-pooling is generalised as follows:

$$P f(n) := \max_{k \in gU} f(k),$$

where $U \subset G$ and $gU = \{gu \,|\, g \in G, u \in U\}$. This definition alone, however, does not guarantee equivariance. Cohen and Welling resolve this issue by using the unique behaviour of groups - in particular, they show that they obtain equivariance by requiring U to itself be a group when paired with the binary operation of G (i.e., by requiring U to be a subgroup of G). When doing this, we find that for every $g_1, g_2 \in G$, we either have that $g_1 U = g_2 U$ or $g_1 U \cap g_2 U = \varnothing$; and also that $G = \bigcup_{g \in G} gU$. In other words, G is equal to the disjoint union of U and its transformations gU (referred to as [left] cosets); U and its cosets evenly divide G into disjoint subsets [3]. This is relevant, as the notion of pooling with stride can be naturally extended by pooling with a pooling domain H which is a subgroup of G. Therefore, we can pool with stride while maintaining equivariance.

5.4 Efficient implementation

As discussed in the paper, computing the G-convolution for discrete groups mainly involves indexing operations and inner products [1]. The key implementation idea is that the groups used in the paper, such as $p4$ and $p4m$, can be decomposed into a spatial translation and an orientation-changing transformation about the origin, such as a rotation or a reflection. This allows the group correlation to be rewritten in a form where the filter is first transformed into all required rotated or reflected versions, after which a standard fast planar convolution can be applied.

To understand this more intuitively, it is useful to first consider a standard CNN. In a normal convolution layer, a filter is a small grid of learned values, such as a 3×3 or 5×5 pattern, that is slid across the image to detect the same feature at different spatial locations. In this case, the built-in symmetry is translation only.

In a $p4$ layer, however, each learned filter is associated with four rotated copies, corresponding to the four 90° rotations. In a $p4m$ layer, each filter is associated with eight transformed copies, since reflections are also included. As a result, the hidden representation is no longer just a single 2D feature map, but a stack of feature maps, one for each allowed orientation state.

This can also be understood in terms of the filter tensor having extra dimensions that encode these symmetry states. Rather than learning every rotated or reflected filter independently, the implementation generates these transformed copies efficiently through inexpensive index permutations. The expanded filter bank can then be reshaped and passed into a standard optimized planar convolution routine. In this way, the network preserves information about both spatial position and orientation, while still reusing the highly optimized convolution operations developed for ordinary CNNs.

5.5 Experimental results

The paper evaluates G-CNNs on two benchmarks: rotated MNIST and CIFAR-10. The purpose of these experiments was to determine whether replacing standard planar convolutions with group convolutions could improve classification performance while keeping the comparison approximately fair in terms of parameter count. On rotated MNIST, the dataset contained 62,000 randomly rotated handwritten digits and was split into 10,000 training, 2,000 validation, and 50,000 test examples. Model selection was performed using the validation set. The baseline model, denoted Z2CNN, used seven convolution layers with mostly 3×3 kernels, a 4×4 final layer, 20 channels per layer, ReLU activations, batch normalization, dropout, and max-pooling after layer 2. The model was optimized using Adam. The authors then replaced each standard convolution with a p4-convolution, reduced the number of filters by a factor of 2 to keep the parameter count approximately fixed, and applied max-pooling over rotations only after the final convolution layer. This model, called P4CNN, achieved substantially lower error than the baseline. They also evaluated a P4CNNRotationPooling variant, in which rotation pooling was applied after each convolution layer, to test whether introducing invariance too early would be harmful. This model improved over the baseline but performed worse than the full P4CNN, supporting the idea that equivariant structure should be preserved deeper in the network.

For CIFAR-10, the dataset was divided into 40,000 training, 10,000 validation, and 10,000 test examples. Two baseline architectures were used: All-CNN-C and ResNet44. To create the equivariant variants, all standard convolutions were replaced with $p4$ or $p4m$ convolutions. Because these group convolutions expand the internal representation, the number of filters was reduced so that the number of parameters remained approximately invariant. Specifically, $p4$ layers used about half as many filters, while $p4m$ layers used about one third as many. The authors also compared performance with and without moderate data augmentation using horizontal flips and small translations. For the ResNet experiments, training used SGD with initial learning rate 0.05, momentum 0.9, and learning-rate reductions at epochs 50, 100, and 150, for a total of 300 epochs. Across both architectures, the equivariant models consistently outperformed their standard counterparts, with p4m-ResNet44 achieving particularly strong results on both plain and augmented CIFAR-10.

6 Discrepencies Between Our Results and the Paper

For the rotated-MNIST experiment, batch normalization is not a major discrepancy because Cohen and Welling also used batch normalization in their reported setup. However, our implementation still differs in how it is embedded within the overall architecture, since we used a different backbone and different layer arrangement. So the discrepancy is less that we used batch normalization at all, and more that our full model design was not identical to theirs.

For CIFAR-10, batch normalization is more reasonable to mention as part of the implementation differences. Our simplified All-CNN-style models apply batch normalization throughout the network, but this does not exactly reproduce the paper's full architecture and training recipe. In other words, even when we use the same general technique, its placement, interaction with the rest of the model, and

effect on training may differ from the original paper. Moreover, we chose to use batch normalization because, without it, the regular $Z2$ CNN would stagnate at around 10% accuracy per epoch.

7 Reproduction Experiment on rotated MNIST

A custom Rotated-MNIST dataset was created from MNIST by assigning each image a fixed random rotation. The same rotated split, optimizer, and checkpoint selection rule were used for all models to ensure a fair comparison. To evaluate the effect of equivariance on image classification, four models were compared: a standard $\mathbb{Z}^2$ CNN, a $p4$ group-equivariant CNN, a $p4m$ group-equivariant CNN, and an early-invariance $p4m$ ablation model. All models were trained on the same custom Rotated-MNIST dataset generated from MNIST, using identical train, validation, and test splits. This ensured that performance differences could be attributed primarily to the choice of symmetry-aware architecture rather than changes in the data or training procedure. For each model, training and validation accuracy and loss were recorded over epochs, and the best checkpoint was selected using validation accuracy. Final comparisons were made using test accuracy, parameter count, confusion matrices, and qualitative prediction examples. This design allowed both quantitative and qualitative analysis of how equivariance affected performance under rotated inputs [1].

7.1 Goal

The goal of this reproduction experiment was not to exactly duplicate the original seven-layer rotated-MNIST architecture used by Cohen and Welling, but rather to reproduce the central hypothesis of the paper on a newer four-stage backbone. Specifically, the experiment tested whether replacing ordinary convolutions with group-equivariant convolutions would improve classification performance on rotated handwritten digits, and whether preserving equivariant structure until late in the network would outperform introducing invariance too early.

7.2 Experimental setup

The first of our four models was a conventional $\mathbb{Z}^2$ CNN baseline with four convolutional stages, batch normalization, ReLU activations, one spatial max-pooling stage after the second convolution, global average pooling, and a final linear classifier. The second model, p4, replaced standard convolutions with equivariant convolutions defined over the discrete rotation group with $N = 4$, making the model equivariant to translations and 90° rotations. The third model, p4m extended this design to the p4m group, making the network equivariant to translations, 90° rotations, and reflections. In both p4 and p4m, group pooling was applied only near the end of the network so that hidden layers could preserve symmetry-aware intermediate features. Finally, an early-invariance ablation model, denoted p4m_RP, applied group pooling after every block, forcing invariance much earlier in the network. This provided a direct way to test whether preserving equivariant structure deeper in the network was beneficial.

7.3 Results

Table 1: Test accuracy on the custom Rotated-MNIST benchmark.

Model	Params	Test Accuracy (%)	Std (%)
$Z2$	207,562	94.09	0.07
$P4$	91,050	96.81	0.05
$P4M$	102,082	94.35	0.04
$P4M_RP$	14,000	89.54	0.08

The results show a clear advantage for the rotation-equivariant model. The strongest finding is that the $P4$ model clearly outperformed all other variants. Compared with the baseline $Z2$ CNN, $P4$ improved test accuracy by 2.72%. This suggests that explicitly encoding rotational equivariance into the network provides a strong inductive bias for rotated digit recognition. Since the task contains large rotational variation, the ability of the $P4$ model to preserve rotation-aware structure through the hidden layers appears to be highly beneficial.

The comparison between $P4M$ and $P4M_RP$ is also informative. Although the late-pooled $P4M$ model only slightly improved over the baseline, it still substantially outperformed the early-pooled $P4M_RP$ variant. This supports the same qualitative conclusion emphasized in the paper introducing invariance too early can reduce performance. In the early-pooled model, group pooling was applied after every block, so later layers no longer had access to the full symmetry-aware intermediate representation. As a result, useful information about orientation and reflection was discarded too soon, limiting the model's ability to build richer hierarchical features. The fact that $P4M_RP$ still achieved close to 90% accuracy despite its much smaller effective capacity suggests that the symmetry prior remains useful, but that preserving group structure deeper in the network is important for maximizing accuracy.

Another notable observation is that $P4M$ did not match the performance of $P4$. A likely explanation is that reflections are a stronger prior than rotations alone, and reflection symmetry is not always naturally label-preserving for handwritten digits. While rotations clearly appear in the task distribution, reflections may sometimes distort class-specific structure in a way that is less helpful for recognition. This suggests that adding more symmetry is not automatically better; the imposed symmetry should align closely with the true invariances of the data.

7.4 Discussion

This experiment has several limitations that should be acknowledged. First, it does not use the exact rotated-MNIST benchmark from Cohen and Welling, but instead constructs a custom rotated dataset from standard MNIST. Although this still provides a meaningful controlled comparison, the numerical results are not directly comparable to the paper's reported benchmark values.

Second, the models are only approximately parameter-matched rather than exactly matched. In particular, the $P4M_RP$ model has substantially fewer effective parameters because early group pooling converts intermediate representations back to trivial feature types, reducing the complexity of later layers. This means that part of the comparison is also a tradeoff between representational richness and model size, rather than a perfectly controlled architectural substitution. This is in line with what the paper suggests in testing the hypothesis of premature invariance in a deep architecture.

Thirdly, our test runs were limited to about 50 epochs and learning rate step decay at 25 epochs which can be clearly seen in the figure X attached in the glossary. The experiments training and validation loss vs epochs tells us that our model would benefit from longer runs and better suited step rates which could have improved our results.

Finally, although the experiment captures the paper's main conceptual comparison between standard, equivariant, and early-invariant models, the backbone architecture differs from the seven-layer architecture used in Cohen and Welling Therefore, the present results are best interpreted as a controlled reproduction of the main idea of the paper on a newer four-stage architecture, rather than as an exact implementation of the original rotated-MNIST protocol.

8 Reproduction Experiment: CIFAR-10

8.1 Goal

Here, we follow a similar experiment to the one in the previous section, but instead use the CIFAR-10 dataset. Three different models are compared: a baseline $\mathbb{Z}^2$ CNN, a $p4$ G-CNN, and a $p4m$ G-CNN.

8.2 Experimental Setup

All three models were trained on the CIFAR-10 dataset. We opted to use 30 epochs, rather than 50, as is done in the previous experiment. After training the models, we compare the validation accuracy of the three models at each epoch, as well as the final test accuracy.

8.3 Results and Discussion

Table 2: Test accuracy on the CIFAR-10 benchmark.

Model	Params	Test Accuracy (%)
$Z2$	1,372,254	87.5
$P4$	926,304	79.32
$P4M$	824,896	76.45

(Refer to Figure 2 in the glossary.) As seen above, the regular CNN both appears to have a higher final test accuracy, and as well as higher validation accuracy per epoch, compared to the G-CNNs. This contrasts what Cohen and Welling found, where the $p4m$ model outperformed the other two models. There are a few potential explanations for this. Firstly, as may be observed from Table 2, the $Z2$ CNN model has far more parameters compared to the other two models. Thus, it is likely able to learn and classify more quickly than the other two models. Cohen and Welling also mention that the CIFAR-10 dataset is not ideal for G-CNNs - much of the dataset is not symmetric [1]. Interestingly, as is seen from Figure 2, the G-CNN models' validation accuracy tends to increase at a faster rate than the regular CNN. This suggests that, despite the larger number of parameters, being equivariant to rotations (and also reflections, in the case of the $P4M$ CNN) allows for models to learn patterns more efficiently. Moreover, the validation accuracy of all three models also tends to become approximately constant at around 11 epochs. Finally, we found that batch normalisation improved the validation accuracy of all three models, though it was a more substantial increase for the $Z2$ CNN, as previously mentioned.

9 Additional Experiment: Rotation Robustness on CIFAR-10

9.1 Goal

This experiment evaluates the rotation robustness of three architectures in a standard CNN (Z^2), a GCNN ($p4$), and a GCNN ($p4m$) on CIFAR-10, by measuring test accuracy across a range of fixed rotation angles. The goal is to empirically verify whether the inductive bias introduced by group equivariance produces representations that degrade more gracefully under unseen rotational transformations.

9.2 Experimental setup

All three models were trained on augmented CIFAR-10 and evaluated at fixed test-time rotation angles: 0°, 45°, 60°, 90°, 120°, 135°, 150°, and 180°. The accuracy gap reflects the parameter reduction applied to the equivariant models to keep comparisons approximately fair.

Table 3: Model comparison on rotated CIFAR-10.

Model	**Parameters**	**Best Test Accuracy**
CNN ($\mathbb{Z}^2$)	1,372,254	91.44%
GCNN (p4)	926,304	88.06%
GCNN (p4m)	824,896	86.27%

9.3 Results and Discussion

(Refer to Figure 3 in the Glossary.) As shown in Figure 3, the CNN (Z^2) degrades substantially as rotation angle increases, reflecting its lack of built-in rotational symmetry. GCNN ($p4$) remains stable at 90° and 180°, consistent with its theoretical equivariance guarantee, and degrades more gradually at non-equivariant angles. This supports a key insight from Cohen and Welling: G-convolution forces the network to share weights across orientations, acting as a regulariser that makes learned features more orientation-general even at angles outside the discrete symmetry steps. GCNN ($p4m$) follows a similar pattern, with modest additional stability from its reflection equivariance. Overall, although

the CNN leads, the equivariant models demonstrate clearly superior robustness under distribution shift which suggests the accuracy gap on standard CIFAR-10 understates the practical advantage of group-equivariant architectures.

10 Additional Experiment: Filter Visualization

10.1 Goal

This experiment analyzes the impact of group equivariance on feature learning in convolutional neural networks. In particular, we want to compare a standard $\mathbb{Z}^2$ CNN with a $p4$ GCNN and a $p4m$ GCNN to see whether the equivariant models learn more structured and orientation-aware filters. By visualizing the learned filters, we can properly assess how symmetry constraints influence the learning of visual patterns.

10.2 Experimental Setup

To visualize the learned first-layer convolution filters from our trained $\mathbb{Z}^2$, $p4$, and $p4m$ models, we are going to use CIFAR-10 as our data set. For each network, the first layer kernels were extracted after training so that they can be displayed as heat maps to show the difference in spatial structures across different architectures. We chose the first layer because it is the point at which the model first transforms image data into learned features, and so it is a favorable place to visualize how each architecture encodes edges and patterns. In the following graphs, we can see that the filter banks emphasize qualitative differences in regularity, orientation sensitivity, and symmetry between the models. Each model was tested on rotations and reflections of 90 degree angles.

10.3 Results and Discussion

(Refer to Figures 4, 5, and 6 in the Glossary.) The visualizations show clear differences between the standard CNN and the group-equivariant models. In the $\mathbb{Z}^2$ CNN, many first-layer filters appear irregular and largely independent, with localized responses but little obvious shared structure. In contrast, the$p4$GCNN filters are more organized and directional, with clearer orientation-selective patterns across related kernels. The $p4m$ GCNN shows the strongest symmetry structure, with more balanced spatial arrangements and more consistent high- and low-response regions, suggesting that both rotation and reflection are shaping the learned features. Overall, the equivariant models produce filter banks that appear more structured than those of the baseline CNN.

These observations are consistent with the central claim of group-equivariant convolutional networks. In a standard CNN, rotated or reflected versions of the same pattern are often learned separately, which can lead to redundancy and less structured filters. By contrast, the $p4$ and $p4m$ models encourage weight sharing across transformed versions of the same feature, so the learned filters more naturally reflect the symmetries built into the architecture. Visually, this appears as rotational consistency across groups of four filters in $p4$, and across groups of eight filters in $p4m$, where both rotations and reflections are represented.

The visualizations also suggest that model quality should not be judged by parameter count or final accuracy alone. Even though the standard CNN has more parameters and strong validation accuracy, its filters appear less structured and more redundant. This indicates that the symmetry prior can improve the quality and efficiency of learned representations, even when it does not always produce the highest absolute accuracy. In this sense, the filter visualizations support the broader idea behind GCNNs: the right geometric inductive bias can lead to more efficient feature learning and representations that better match the structure of the data.

11 Conclusion

This report examined the main ideas of Cohen and Welling's Group Equivariant Convolutional Networks (G-CNNs) and evaluated how well those ideas hold in practice through our reproduction experiments. The paper's central contribution is the extension of standard convolutional neural networks so that they are not only translation-equivariant, but also equivariant to richer symmetry

groups such as rotations and reflections. This provides a more principled alternative to relying entirely on data augmentation, while also increasing the degree of parameter sharing within the network.

Our reproduction experiments showed that these ideas can provide meaningful benefits when the chosen symmetry matches the structure of the dataset. On the custom Rotated-MNIST task, the $P4$ model achieved the strongest performance, outperforming the standard $Z2$ baseline and supporting the claim that rotational equivariance is a useful inductive bias for rotated digit recognition. The weaker performance of the early-pooled $P4M_RP$ variant also reinforced one of the paper's important insights: introducing invariance too early can reduce performance by discarding useful intermediate structure.

At the same time, our CIFAR-10 experiments highlighted that equivariance is not automatically beneficial in every setting. In our implementation, the standard CNN achieved higher final accuracy than the equivariant variants, suggesting that the usefulness of added symmetry depends strongly on the task, the dataset, and the architectural choices used in training. This emphasizes that group-equivariant methods should be viewed as targeted tools rather than universal replacements for standard CNNs.

Overall, this project showed that the main value of G-CNNs lies in aligning model structure with data symmetry. While our experiments were not exact reproductions of the original benchmarks, they still supported the broader conclusion that carefully chosen symmetry-aware architectures can improve learning efficiency, robustness, and generalization.

12 Reflections

Kavin Kumaran

I learned that group equivariance is a powerful inductive bias enabling encoding symmetries like rotation and reflection directly into the architecture, so the network achieves better generalization with fewer parameters than a standard CNN. The additional rotation robustness experiment taught me that equivariant models degrade more gracefully under distribution shift, particularly at angles like 90 and 180 degrees where the symmetries hold exactly. Additionally, from the class presentation on neural collapse, I learned that when a network is trained past zero error, its last-layer features naturally organize into a highly symmetric structure, which improves robustness and generalization without any explicit design choices to make it do so.

Adil Tapal - Reflection

From this project, I learned that Equivariance is most useful when it matches the real structure of the data and to reproduce any good results you need to first understand what you are solving in depth. Additionally, from the class presentation on the "Toy models of superposition", I learn't how modern are at the same time juggling with many ideas and not just one key extract idea, which makes the model more efficient but harder to understand and link together.

Robin Liu

The presentation on Superposition Disentanglement by my friend Michael gave me a much better understanding of how neural networks can store multiple concepts in overlapping representations. I learned that this superposition can make a model's internal behavior harder to interpret, since a single representation may contain several mixed signals at once. The idea of disentangling those representations helped me see how interpretability methods can separate these overlapping features and reveal structure that is not obvious from the raw outputs alone.

It also gave me a stronger mental model for thinking about convolutional layers and learned filters. I began to see that filters may not always respond to just one simple pattern, but can instead reflect several interacting visual features that become entangled during training. This helped me better understand why analyzing filters and intermediate activations is important when trying to explain what a convolutional network has actually learned. More broadly, I learned that interpretability is not only about visualizing results, but about uncovering how internal representations are organized and how different learned phenomena influence the behavior of the network.

Most of the learning process for our paper was centered around understanding group equivariance and for me, especially the coding portion since I mainly handled all the code and instructed my teammates on what/how to structure the Git repo and even their own experiments. Through exploring the topic, I was able to develop a mental model of what Neural Networks are structurally. Additionally, the experiment on Filter Visualization also showed to me that performance metrics should not be the only way we evaluate models. Through visualizing, the filters on $p4$ and $p4m$ I was able to discover why my models got worse accuracy despite supposedly being 'better' and why the performance metrics, in a sense, were fake when it comes to describing the performance and efficiency of our models.

Hasan Zia

I found this project very useful for several reasons. Firstly, group theory is a field in math that I personally find very interesting. I have studied it at the university (I took MATH 328, an intro course on group theory), and plan on studying it further in the future. This project was the first real opportunity I had to explore the applications of group theory. I believe group theory can and should be applied more regularly in our fields of engineering and programming - since it studies symmetry, a fundamental concept in math, I believe there are many uses and applications for it that have not yet been found. In the case of Cohen and Welling, applying it in the context of equivariance is a natural application of the study. I also found it neat how the concept of subgroups and cosets, and the even subdivision it provides, came up when implementing pooling with stride. Moreover, the other presentations also allowed me to gain insight into the behaviour of neural networks. For instance, I learnt that one area of study appears to be in the simplification of neural networks, which was discussed by papers on the Lottery Ticket Hypothesis and Neural Collapse. I find this topic interesting, and believe methods from Automata Theory may come useful in this regard.

13 References

[1] T. Cohen and M. Welling, “Group Equivariant Convolutional Networks,” Feb. 2016.
[2] B. Tolooshams. 2026. ECE 447 Lecture 14: Neural Networks [PowerPoint Slides].
[3] N. Guay. 2024. MATH 328 Fall 2024 Notes for Students [PDF].

14 Glossary

Figure 1: Training and validation loss curves for the four compared models on custom Rotated-MNIST. The $P4$ model achieves the lowest validation loss and the most stable convergence behavior.

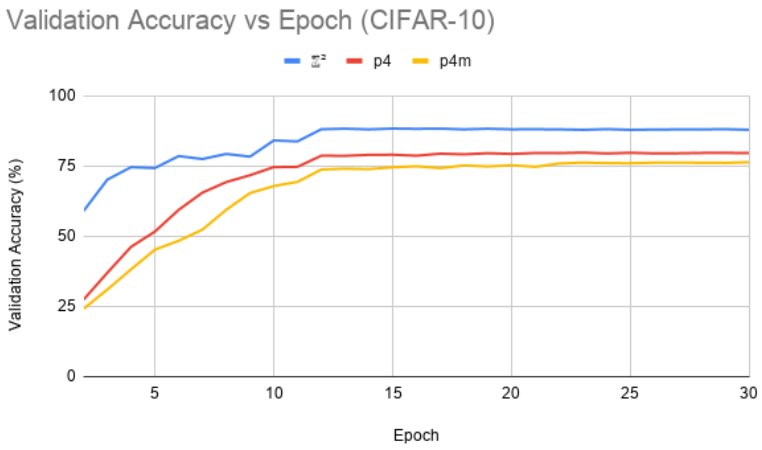

Figure 2: Validation accuracy curves for the three models trained on CIFAR-10. Unlike what Cohen and Welling find, the regular $Z2$ CNN appears to consistently outperform the G-CNNs.

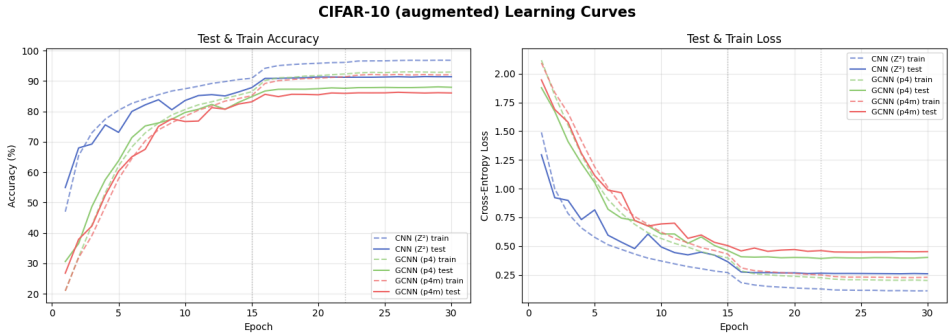

Figure 3: Test accuracy of CNN ($\mathbb{Z}^2$), GCNN ($p4$), and GCNN ($p4m$) on CIFAR-10 as a function of test-time rotation angle. All models were trained on augmented data and evaluated at each fixed angle without rotational training. The equivariant models degrade more gracefully than the CNN baseline, particularly at 90° and 180°.

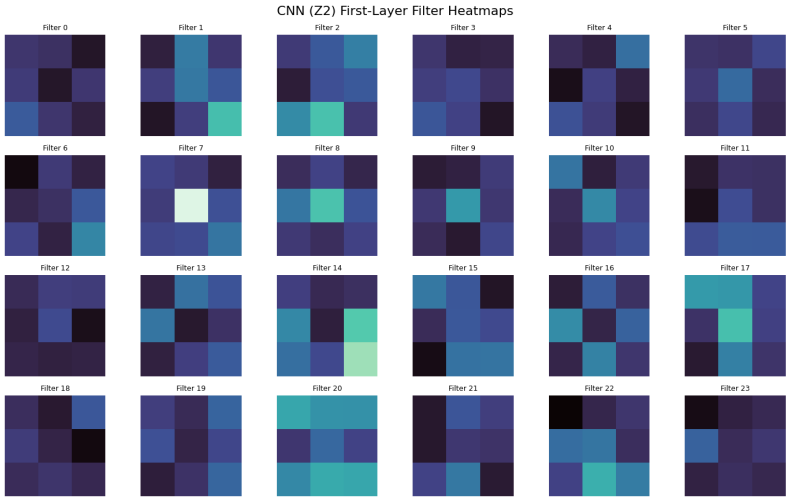

Figure 4: Heatmaps of the first-layer filter of the regular $Z2$ CNN.

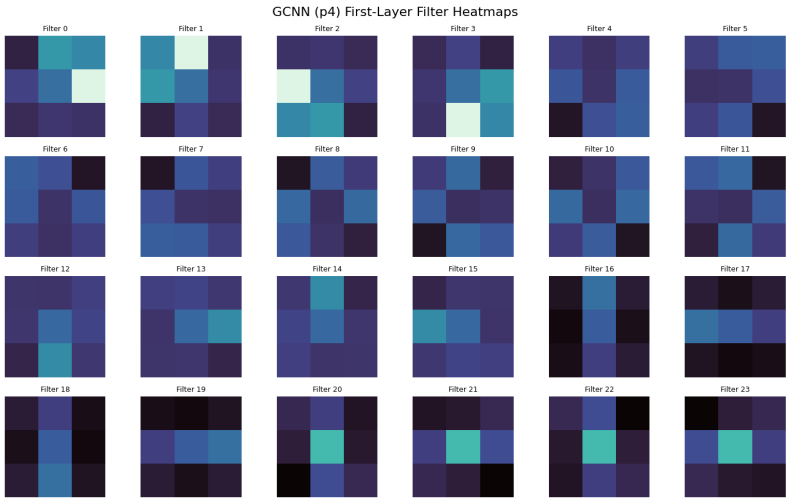

Figure 5: Heatmaps of the first-layer filter of the $p4$ G-CNN.

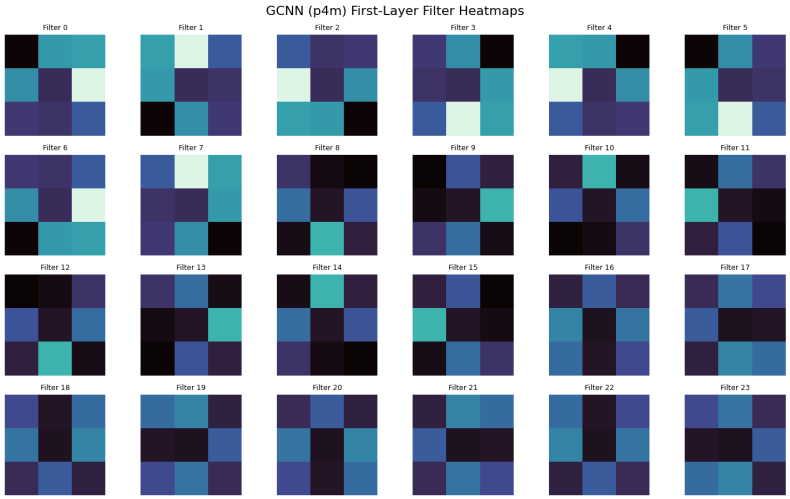

Figure 6: Heatmaps of the first-layer filter of the $p4m$ G-CNN.